

VentaGO 프로젝트 구조 및 동작 방식

1. 프로젝트 개요

VentaGO는 상점(매장)의 판매, 재고, 상품 관리를 위한 POS/ERP 시스템입니다. 이 프로젝트는 npm workspaces를 사용하는 모노레포 구조로 구성되어 있으며, 백엔드와 프론트엔드가 하나의 저장소에서 관리됩니다.

2. 아키텍처 구조

2.1 모노레포 구조

프로젝트는 npm workspaces를 사용하여 두 개의 워크스페이스로 구성됩니다:

워크스페이스	기술 스택	포트	설명
api-ventago	NestJS + PostgreSQL	5002	백엔드 API 서버
ventago-app	Next.js + React	3000	프론트엔드 웹 애플리케이션

3. 백엔드 구조 (api-ventago)

3.1 기술 스택

- NestJS 11.x - Node.js 프레임워크
- TypeScript - 타입 안정성
- PostgreSQL - 관계형 데이터베이스
- Sequelize - ORM (Object-Relational Mapping)
- JWT - 인증 토큰
- Passport - 인증 전략

3.2 주요 모듈

- Auth - 사용자 인증 및 권한 관리
- Users - 사용자 관리
- Store - 매장 관리
- Products - 상품 관리
- Sales - 판매 관리
- Stock - 재고 관리

- Clients - 고객 관리
- Role - 역할 및 권한 관리
- Audit Log - 감사 로그

3.3 데이터베이스 동기화

애플리케이션 시작 시 SyncService가 모든 모델을 순회하며 데이터베이스 테이블을 자동으로 생성합니다. 각 테이블 생성 시 테이블 이름만 간단하게 출력됩니다.

4. 프론트엔드 구조 (ventago-app)

4.1 기술 스택

- Next.js 13.x - React 프레임워크
- React 18 - UI 라이브러리
- Material-UI (MUI) - UI 컴포넌트
- Redux Toolkit - 상태 관리
- TypeScript - 타입 안정성
- Axios - HTTP 클라이언트

4.2 주요 기능

- 사용자 인증 및 권한 관리
- 매장 관리 대시보드
- 상품 및 재고 관리
- 판매 처리 및 관리
- 고객 관리
- 보고서 및 분석

5. 데이터 흐름

5.1 인증 흐름

1. 사용자가 로그인 페이지에서 이메일/사용자명과 비밀번호 입력
2. 프론트엔드가 POST /api/auth/login 요청 전송
3. 백엔드에서 사용자 검증 및 JWT 토큰 생성
4. 토큰을 localStorage에 저장
5. 이후 모든 API 요청에 Authorization 헤더로 토큰 포함

5.2 API 요청 흐름

1. 프론트엔드 컴포넌트에서 apiConnector 사용
2. Axios 인터셉터가 요청에 토큰 자동 추가
3. 백엔드 NestJS 컨트롤러에서 요청 수신
4. Guard를 통한 인증/권한 검증
5. Service에서 비즈니스 로직 처리
6. Sequelize를 통해 데이터베이스 조작
7. Audit Interceptor가 변경 사항 기록
8. 응답 반환

6. 실행 방법

6.1 개발 환경 설정

1. 의존성 설치: `npm install`
2. 데이터베이스 설정: PostgreSQL 설치 및 데이터베이스 생성
3. 환경 변수 설정: `api-ventago/.env` 파일 생성
4. 마이그레이션 실행: `npm run db:migrate`

6.2 개발 서버 실행

두 프로젝트를 동시에 실행:

```
npm run dev
```

개별 실행:

```
npm run dev:api # ■■■■  
npm run dev:app # ■■■■■■
```

7. 주요 디렉토리 구조

7.1 백엔드 (api-ventago/src)

- app/ - 비즈니스 로직 모듈들
- auth/ - 인증 관련
- users/ - 사용자 관리
- products/ - 상품 관리
- sales/ - 판매 관리
- store/ - 매장 관리
- common/ - 공통 유틸리티
- config/ - 설정 파일
- database/ - 데이터베이스 연결 및 동기화

7.2 프론트엔드 (ventago-app/src)

- pages/ - Next.js 페이지 라우팅
- views/ - 주요 뷰 컴포넌트
- components/ - 재사용 가능한 컴포넌트
- services/ - API 서비스
- store/ - Redux 상태 관리
- layouts/ - 레이아웃 컴포넌트
- hooks/ - 커스텀 React 훅

8. 보안 및 권한 관리

8.1 인증

JWT 토큰 기반 인증을 사용합니다. 로그인 시 accessToken과 refreshToken이 발급되며, 모든 API 요청에 Authorization 헤더로 토큰이 포함됩니다.

8.2 권한 관리

역할 기반 접근 제어(RBAC)를 구현합니다. 사용자는 역할(Role)을 가지며, 각 역할은 여러 기능(Function)에 대한 권한을 가집니다. 프론트엔드에서는 WithFunctionAccess HOC를 사용하여 권한에 따른 UI 제어를 합니다.

9. 감사 로그 (Audit Log)

모든 중요한 데이터 변경 사항은 AuditInterceptor를 통해 자동으로 기록됩니다. 생성, 수정, 삭제 작업이 발생할 때마다 사용자, 시간, 변경 내용이 audit_logs 테이블에 저장됩니다.

10. 요약

- 모노레포 구조로 백엔드와 프론트엔드를 통합 관리
- NestJS와 Next.js를 사용한 풀스택 TypeScript 프로젝트
- PostgreSQL과 Sequelize를 사용한 데이터베이스 관리
- JWT 기반 인증 및 역할 기반 권한 관리
- 자동화된 감사 로그 시스템
- 개발 환경에서 두 서버를 동시에 실행 가능